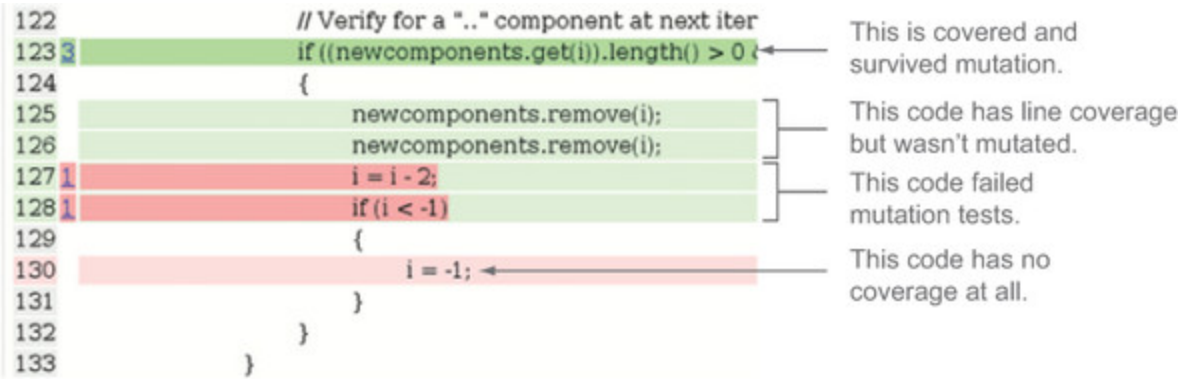
code, it’s not really testing your code. A great tool for this is Pitest (pitest.org/). An example screenshot of a Pitest analysis is shown in figure 8.10.

Figure 8.10. An example mutation test report using Pitest



Another strategy for vetting your code coverage numbers is to track it alongside other metrics. When your codebase has a great suite of automated tests, that usually translates into a fix release percentage and a maintainable release rating. If you have great coverage but your releases aren’t going well, that’s a good sign that either you’re not testing the right things or your tests are bad.

Often code coverage gets lumped in with static code analysis because the two reports are typically generated at the same time. The two are complementary but different measurements of a maintainable codebase.

8.3.4. Adding static code analysis

Static code analysis can check your code for best practices against common rule sets for whatever language you’re using. A number of tools can do this for you. SonarQube is a great option.

As I mentioned in chapter 5, *SonarQube in Action*, published by Manning, is a great book with a lot of details, but I’ll give you a quick rundown of maintainability highlights that you can start using today.